CHMLTECH ONLINE JOURNAL · VOLUME 104, ISSUE 1 · RESEARCH ARTICLE

AI Operating Systems: A Reference Architecture and Governance Model for Enterprise AI

Quak Foo Lee
Chief Technology Officer, Chmltech Ltd.
Division of Artificial Intelligence and Machine Learning · Vancouver, BC, Canada

Published November 9, 2026 21 min read

ABSTRACT

Organizations that adopt large language models (LLMs) through isolated assistants and pilots accumulate fragmented prompts, data connections, credentials and costs that are difficult to govern. This paper develops the concept of an AI operating system (AI OS): a shared, organization-wide layer that mediates access to data, knowledge, models, tools and workflows, in the way that a conventional operating system mediates access to hardware and files. It reviews research that treats LLM runtimes as operating systems, notably MemGPT and AIOS, together with work on tool use, retrieval augmentation and the Model Context Protocol. A ten-layer reference architecture is proposed, covering data and integration, knowledge and retrieval, model access and routing, agents and tools, workflow orchestration, memory, identity and permissions, governance, evaluation and observability, and cost control, with the function, mechanisms and principal risks of each layer. The architecture is mapped to the NIST AI Risk Management Framework, ISO/IEC 42001:2023, the EU Artificial Intelligence Act and the OWASP Top 10 for LLM Applications 2025. Operating metrics are defined for routing, cost per successfully completed

task, automation rate and end-to-end reliability, and a five-stage maturity model is proposed. A hypothetical SME invoice-processing example shows that human review can dominate total cost and that reliability improvements may matter more than cheaper models. The paper concludes that mediation of all model, tool and data access is the decision on which governance, security and cost control depend.

Keywords: AI operating system; enterprise AI architecture; AI governance; LLM agents; model routing; retrieval-augmented generation; prompt injection; Model Context Protocol

1. Introduction

Most organizations first encounter large language models (LLMs) as isolated tools: a chat assistant used by individual staff, a document summarizer embedded in one application, or a pilot that automates a single workflow. Each such deployment carries its own prompts, data connections, credentials, logging conventions and cost profile. As the number of deployments grows, the organization accumulates a fragmented estate in which the same customer record is retrieved by several pipelines, the same policy is restated in several system prompts, and nobody can answer simple questions such as which models touch personal information or what the AI estate cost last month. The phenomenon is a generative-AI analogue of the hidden technical debt that Sculley et al. described for conventional machine learning systems, where the model code is a small fraction of a much larger body of configuration, data dependencies and glue code [1].

This paper uses the term *AI operating system* (AI OS) for the shared, organization-wide layer that addresses this fragmentation. An AI OS is not a single product. It is an architectural pattern: a set of common services for data access, knowledge retrieval, model access and routing, tool execution, workflow orchestration, memory, identity and permissions, governance, evaluation, observability and cost control, on top of which individual AI applications and agents are built. The analogy to a conventional operating system is deliberate. An operating system abstracts hardware, schedules processes, isolates them from one another, mediates access to files and devices, and enforces permissions; an AI OS performs corresponding functions for models, agents, tools and enterprise data.

The analogy has research precedent. MemGPT treats the LLM context window as a scarce fast memory and manages movement of information between it and slower external storage, in the manner of virtual memory [2]. AIOS proposes an operating-system kernel for LLM agents that provides scheduling, context management, memory and storage management and access control as shared services [3]. These systems operate at the level of a single runtime, whereas the present paper extends the analogy to the enterprise level, where the dominant concerns are integration with existing systems, accountability and risk.

The contribution of the paper is threefold. First, it proposes a layered reference architecture for an enterprise AI OS and identifies, for each layer, its function, typical mechanisms and principal risks. Second, it maps the architecture onto published governance frameworks and security guidance and defines a small set of quantitative operating metrics. Third, it proposes a staged maturity model and illustrates the metrics with a clearly hypothetical worked example for a small or medium-sized enterprise (SME). Section 2 reviews related work; Section 3 presents the architecture; Section 4 addresses governance, evaluation and observability; Section 5 treats security; Section 6 presents the maturity model; Section 7 gives the worked example; Section 8 discusses limitations; and Section 9 concludes.

2. Background and Related Work

2.1 The LLM-as-operating-system analogy

MemGPT addresses the limited context window of LLMs through what its authors call virtual context management. The model is given a hierarchy of memory tiers, a bounded main context that it reads directly and external stores that it can search and write through function calls, and it uses interrupts to manage control flow between itself and the user. The authors evaluated the design on analysis of documents far larger than the underlying context window and on multi-session conversational agents that retain information across sessions [2]. The relevant lesson for enterprise architecture is that memory is a managed resource with explicit policies for what is kept in context, what is paged out and what is persisted.

AIOS approaches the analogy from the systems side. It isolates LLM and tool resources from agent applications in an AIOS kernel, which provides scheduling, context management, memory management, storage management and access control to running agents, and exposes those services through a software development kit. The authors reported up to 2.1× faster execution when serving agents built with several agent frameworks on AIOS, which

they attribute to scheduling and resource management [3]. For an enterprise, the transferable idea is that agents should not hold raw, unmediated access to models, tools and data; access should pass through a shared layer that can queue, meter, authorize and log it.

2.2 Tool use and agents

An LLM becomes an agent when it can take actions rather than only produce text. Surveys of LLM-based autonomous agents describe a common anatomy comprising a profile or role, a memory module, a planning module and an action module that invokes tools [4]. ReAct interleaves free-text reasoning traces with actions such as search queries, so that observations returned by the environment inform subsequent reasoning [5]. Toolformer showed that a language model can learn in a self-supervised manner when to call external application programming interfaces (APIs), such as a calculator, a question-answering system, search engines, a translation system and a calendar, and how to incorporate their results [6]. Earlier, the MRKL architecture proposed a modular, neuro-symbolic system in which a router directs each input to the most suitable expert module, which may be a neural model or a symbolic component such as a calculator or database query [7]. Gorilla fine-tuned a model specifically for generating API calls and reported reduced hallucination of API usage compared with general-purpose models [8]. Together these works establish tool invocation as a central, and security-critical, capability of any AI OS.

2.3 Retrieval augmentation

Retrieval-augmented generation (RAG) combines a parametric model with a non-parametric memory, such as a dense vector index of documents, from which passages are retrieved and supplied to the generator [9]. A subsequent survey organizes the field into naive, advanced and modular RAG paradigms, distinguished by the sophistication of indexing, query rewriting, re-ranking and the composition of retrieval and generation modules [10]. Retrieval does not remove the need to manage context carefully: Liu et al. found that model performance can degrade significantly when relevant information is placed in the middle of a long input context rather than at its beginning or end, even for models designed for long contexts [11]. An enterprise knowledge layer therefore needs ranking and context-assembly policies, not merely a large index.

2.4 Interoperability through the Model Context Protocol

A practical obstacle to a shared AI layer has been the absence of a common interface between models and enterprise systems. The Model Context Protocol (MCP), introduced by Anthropic in November 2024 as an open standard for connecting AI assistants to the systems where data resides [12], addresses this obstacle. The specification defines JSON-RPC 2.0 messages exchanged between hosts (LLM applications), clients (connectors within a host) and servers (services that provide context and capabilities). Servers may expose resources, prompts and tools; clients may offer sampling, roots and elicitation to servers [13]. The specification also states security principles, including explicit user consent before tools are invoked and the instruction that descriptions of tool behavior be treated as untrusted unless obtained from a trusted server, while noting that the protocol itself cannot enforce these principles [13]. In AI OS terms, MCP is a candidate standard for the boundary between the agent layer and the integration layer, but enforcement remains the responsibility of the platform.

3. A Reference Architecture

3.1 Design principles

The reference architecture rests on four principles. **Mediation**: agents and applications reach models, tools and data only through platform services that can authorize, meter and log each request, following the AIOS kernel concept [3] and the zero-trust principle that no request is trusted by virtue of its network location [14]. **Separation of concerns**: knowledge, models, tools and workflows are versioned and replaced independently, which limits the entanglement that Sculley et al. identified as a source of technical debt [1]. **Least privilege**: every agent acts under an identity whose permissions do not exceed those of the human or service on whose behalf it acts. **Measurability**: every unit of work produces a trace from which quality, cost and latency can be computed. Table 1 summarizes the resulting layers. Figure 1 arranges the same ten layers as a stack: six lie on the path between applications and systems of record, while identity and permissions, governance, evaluation and observability, and cost control cut across all of them, so every request meets them whichever layers it uses.

Table 1. Reference architecture for an enterprise AI operating system: layers, functions, typical mechanisms and principal risks.

Layer	Primary function	Typical mechanisms	Principal risks
Data and integration	Governed access to systems of record	Connectors, APIs, MCP servers, change-data capture, data contracts	Stale or inconsistent data; over-broad credentials
Knowledge and retrieval	Ground outputs in organizational content	Chunking, embeddings, hybrid search, re-ranking, access-filtered indexes	Leakage across permission boundaries; poor recall; context dilution
Model access and routing	Select and call models	Model gateway, routers, cascades, caching, fallbacks	Vendor lock-in; quality regression after model updates
Agent and tool	Execute actions through tools	Tool registry with schemas, ReAct-style loops, sandboxes	Excessive agency; injected instructions in tool output
Workflow orchestration	Compose steps into durable processes	State machines, queues, retries, human-approval gates	Silent partial failure; compounding error
Memory	Persist context across sessions	Session state, episodic and semantic stores, retention rules	Retention of personal data; memory poisoning
Identity and permissions	Bind every action to an accountable principal	Single sign-on, delegated tokens, role- and attribute-based access control	Privilege escalation; confused-deputy behavior
Governance	Policy, accountability and compliance	AI inventory, risk assessment, model cards, approval workflow	Unregistered shadow deployments
Evaluation and observability	Measure quality, safety, latency	Test suites, LLM-as-judge with human	Unmeasured drift; biased automated

Layer	Primary function	Typical mechanisms	Principal risks
		calibration, traces	judges
Cost control	Allocate and bound spending	Token metering, budgets, quotas, routing policies	Unbounded consumption; unattributed spend

Synthesized by the author from [2, 3, 13, 17, 24].

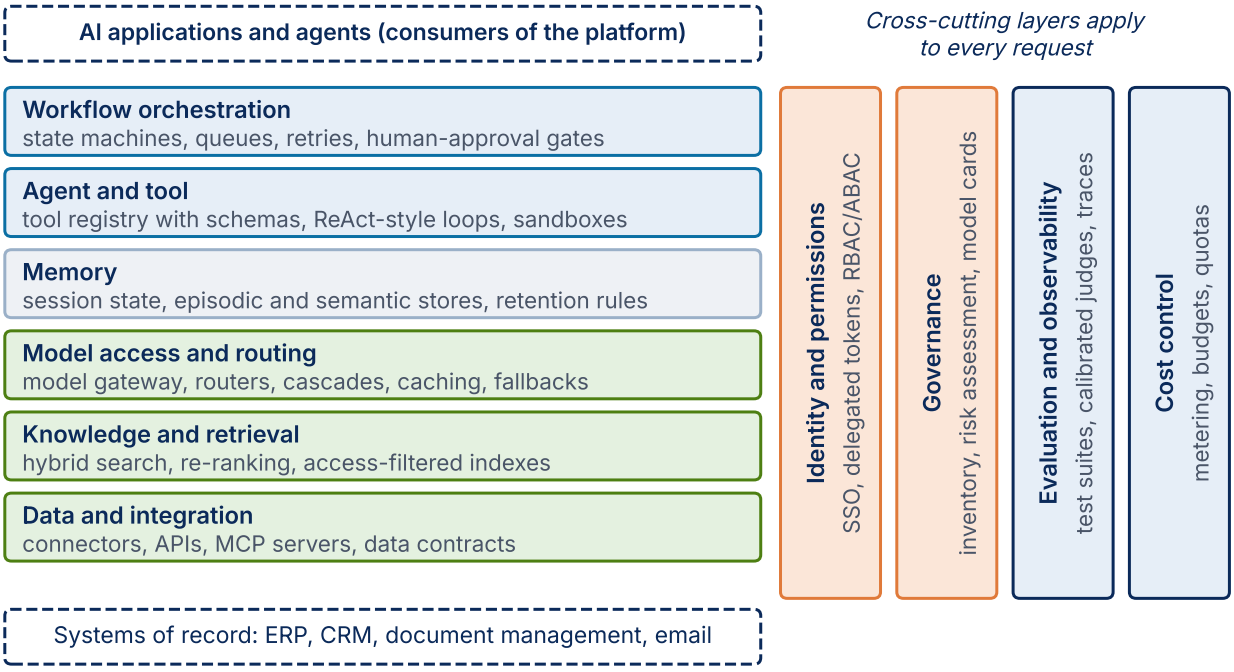


Figure 1. The ten-layer reference architecture of Table 1. Six layers lie on the path from AI applications to systems of record; four cross-cutting layers apply to every request. Typical mechanisms are listed for each layer; the stacked and cross-cutting grouping is the author's.

Drawn by the author from Table 1.

3.2 Data and integration layer

The data and integration layer provides governed, read-and-write access to systems of record such as enterprise resource planning, customer relationship management, document management and email. Its central design decision is whether AI components call systems directly or through intermediary services that enforce data contracts, field-level masking and rate limits. Protocol standards such as MCP reduce the cost of building connectors by giving each system a uniform server interface [13], but the connector inherits whatever credentials it

is given. The layer should therefore issue scoped credentials per connector and per caller rather than a single service account with broad rights.

3.3 Knowledge and retrieval layer

The knowledge layer turns documents, records and transcripts into retrievable context, following the RAG pattern [9]. Enterprise deployments typically need the advanced or modular forms described by Gao et al., including query rewriting, hybrid lexical and vector search and re-ranking [10]. Two enterprise-specific requirements dominate. First, retrieval must respect source permissions: a passage that a user cannot open in the source system must not be retrieved on that user's behalf. Permission filters should be applied at query time against the caller's identity rather than baked into separate indexes that drift out of date. Second, because models use information unevenly across long contexts [11], the layer should return a small number of highly ranked passages with provenance rather than maximizing the volume of retrieved text.

3.4 Model access and routing layer

A model gateway decouples applications from specific model providers and versions. Its most consequential function is routing. FrugalGPT demonstrated that an LLM cascade, which queries inexpensive models first and escalates only when a learned scorer rejects an answer, could match the performance of the best individual model on its benchmarks with up to 98% cost reduction [15]. RouteLLM trained routers on human preference data to choose between a stronger and a weaker model and reported cost reductions of more than a factor of two in some settings without loss of response quality [16]. The MRKL architecture had earlier proposed routing among heterogeneous expert modules [7]. A simple formalization of a threshold router selects, for input x, the cheapest model whose predicted quality meets a threshold:

$$m^*(x) = \arg\min_{m \in M} c_m(x) \text{ subject to } \hat{q}_m(x) \geq \tau \tag{1}$$

where M is the set of available models, $c_m(x)$ is the expected cost of model m on input x, $\hat{q}_m(x)$ is a predicted quality score and τ is a task-specific acceptance threshold. If no model satisfies the constraint, the request escalates to the strongest model or to a human. Eq. (1) makes explicit that a router is only as good as its quality predictor, which must be validated on

the organization's own workload; published savings are workload-dependent and do not transfer automatically [15, 16]. The gateway also implements response caching, fallback on provider outage and pinning of model versions, so that a provider update does not silently change behavior in production.

3.5 Agent and tool layer

The agent layer hosts reasoning-and-acting loops of the ReAct type [5] together with a tool registry. Each registered tool should carry a machine-readable schema, an owner, a risk classification and a declaration of whether it reads, writes or performs irreversible actions. The registry allows the platform to present each agent with only the tools its task requires. Model fine-tuning for tool use can reduce malformed calls [6, 8], but schema validation of arguments at the platform boundary remains necessary because model output is not trustworthy input. Tools that execute code or reach external networks should run in sandboxes with constrained egress.

3.6 Workflow orchestration

Many enterprise tasks are not open-ended conversations but processes with defined steps, such as receiving a document, extracting fields, validating them against a system of record, obtaining approval and posting a transaction. Orchestration encodes such processes as explicit state machines in which some steps are performed by LLM agents and others by deterministic code or humans. Explicit orchestration provides durable state, retries with idempotency, timeouts and human-approval gates, and it confines model nondeterminism to the steps that require language understanding. A useful default is to let deterministic code control the flow and to call models for bounded subtasks, reserving free-form agent autonomy for tasks whose steps cannot be enumerated in advance.

3.7 Memory

Memory spans short-term session state, episodic records of past interactions and semantic stores of distilled facts. MemGPT illustrates explicit management of what enters the limited context and what is paged to external storage [2]. In an enterprise setting memory is also a data-protection concern: persisted memories may contain personal information and must be subject to retention schedules, access control and deletion on request. Memory written by an

agent from untrusted content is also an attack surface, because an injected instruction stored today may be retrieved and acted upon later.

3.8 Identity and permissions

Every action taken by an agent should be attributable to an accountable principal. The zero-trust model, under which access decisions are made per request on the basis of authenticated identity and policy rather than network location [14], fits agent systems well. In practice this implies delegated, short-lived tokens that carry the identity of the requesting user, attribute-based policies that can express conditions such as amount limits, and a prohibition on agents using standing administrative credentials. Without delegation, an agent with broad service credentials can be induced to act as a confused deputy, performing on behalf of one user an action that only another principal is entitled to perform.

4. Governance, Evaluation and Observability

4.1 Governance frameworks

Three instruments provide the main external reference points as of late 2026. The NIST AI Risk Management Framework (AI RMF 1.0) organizes AI risk management into four functions, Govern, Map, Measure and Manage, and is voluntary and sector-neutral [17]. Its Generative AI Profile, published in July 2024, applies the framework to risks that are specific to or amplified by generative AI [18]. ISO/IEC 42001:2023 specifies requirements for establishing, implementing, maintaining and continually improving an AI management system within an organization of any size and is designed for certification [19]. The European Union's Artificial Intelligence Act, Regulation (EU) 2024/1689, entered into force on 1 August 2024 and applies generally from 2 August 2026, with certain provisions applying on other dates set out in Article 113; it establishes risk-based obligations that bear on organizations placing AI systems on the EU market or using them there [20].

An AI OS makes these frameworks operable because it centralizes the evidence they require. The platform's inventory of models, tools, agents and workflows forms the system register that ISO/IEC 42001 and the Map function presuppose; per-deployment documentation in the style of model cards [21] records intended use, evaluation results and known limitations; and traces provide the measurement record for the Measure and Manage functions [17]. Governance implemented as platform policy, for example a rule that no workflow may call a

write-capable tool without a registered owner and a passed evaluation, scales better than governance implemented as periodic manual review.

4.2 Evaluation

Evaluation should be multi-dimensional. The Holistic Evaluation of Language Models (HELM) project argued for measuring accuracy together with calibration, robustness, fairness, bias, toxicity and efficiency across many scenarios, rather than reporting a single score [22]. Enterprise evaluation adds task-specific criteria such as field-extraction accuracy, policy compliance and correct tool selection. Because human grading is expensive, many teams use a strong LLM as an automated judge. Zheng et al. found that a strong judge model could reach agreement with human preferences comparable to agreement between humans, but also documented position, verbosity and self-enhancement biases [23]. Automated judges should therefore be calibrated periodically against human-labeled samples drawn from production traffic, and evaluation suites should be rerun whenever a model, prompt, tool or retrieval index changes.

4.3 Operating metrics

Observability converts traces into a small set of metrics that can be tracked over time. The first is the fully loaded cost per successfully completed task:

$$C_{task} = [\ \Sigma_i\ (\ p_{in,i}\ T_{in,i} + p_{out,i}\ T_{out,i}\ )\ +\ C_{tool} + C_{human}\ ]\ /\ N_{success} \tag{2}$$

where the sum runs over all model calls in a reporting period, $p_{in,i}$ and $p_{out,i}$ are the per-token prices of input and output tokens for call *i*, $T_{in,i}$ and $T_{out,i}$ are the corresponding token counts, C_{tool} is the cost of tool and infrastructure usage, C_{human} is the cost of human review and correction, and $N_{success}$ is the number of tasks completed correctly. Dividing by successful rather than attempted tasks penalizes systems that are cheap per call but fail often. The second metric is the automation rate:

$$A = N_{auto}\ /\ N_{total} \tag{3}$$

where N_{auto} is the number of tasks completed and accepted without human intervention and N_{total} is the number of tasks received. The automation rate must always be reported together with a quality measure, since it can be raised trivially by removing review. The third metric is end-to-end reliability of a workflow composed of k steps with step success probabilities r_j. Under an assumption of independence,

$$R = \Pi_{j=1}^{k} r_j, \; r'_j = 1 - (1 - r_j)^{n_j} \tag{4}$$

where r'_j is the effective success probability of step j when a reliable validator detects failure and permits up to n_j independent attempts. Eq. (4) shows why long autonomous chains are fragile and why cheap validation is valuable. The independence assumption is optimistic: repeated attempts by the same model on the same input tend to fail in correlated ways, so measured retry gains are usually smaller than the formula suggests. Latency percentiles, escalation rates, refusal rates and evaluation scores by workflow complete the operating dashboard.

4.4 Cost control

Cost control combines metering, attribution and limits. Each model call should be tagged with the workflow, business unit and principal that caused it, so that Eq. (2) can be computed per workflow. Budgets and quotas at the gateway bound spending, and routing policies (Eq. (1)) direct routine traffic to inexpensive models. Limits also have a security function: the OWASP Top 10 for LLM Applications lists unbounded consumption as a distinct risk, covering both runaway cost and denial of service [24].

> Cost per successfully completed task, not cost per token, is the economic metric that matters. In workflows with human review, review labor frequently exceeds model spending, so improvements in reliability can reduce total cost more than cheaper models do.

5. Security and Risk

5.1 Prompt injection

Prompt injection is the defining security problem of LLM-integrated systems. Perez and Ribeiro demonstrated goal hijacking and prompt leaking through adversarial user input [25]. Greshake et al. showed that the more serious threat is indirect: an adversary places instructions in content that the application will later retrieve, such as a web page, an email or a document, and the model executes them with the application's privileges. They demonstrated data theft, manipulation of application behavior and control over which APIs are called against real systems, and argued that LLM-integrated applications blur the boundary between data and instructions [26]. An AI OS amplifies this risk because it connects many data sources to many tools; a single poisoned document in the knowledge layer can reach every agent that retrieves it.

No published technique eliminates prompt injection, so the architecture must limit its consequences. Effective measures are structural: separating trusted instructions from untrusted content in prompts and in logs; giving agents that process untrusted content no access to write-capable or exfiltration-capable tools; requiring human approval for irreversible actions; constraining network egress; and validating tool arguments against schemas and policy before execution. The MCP specification's instruction to treat tool descriptions as untrusted unless obtained from a trusted server reflects the same principle at the protocol boundary [13].

5.2 Mapping risks to platform controls

The OWASP Top 10 for LLM Applications 2025 provides a widely used risk catalogue [24]. Table 2 maps a selection of its entries to the architecture layers of Table 1 and to controls that a platform can enforce centrally. The mapping illustrates the main security argument for an AI OS: controls implemented once in the gateway, registry and identity layers apply to every application, whereas controls implemented separately in each application tend to be inconsistent.

Table 2. Selected OWASP Top 10 for LLM Applications 2025 risks mapped to AI OS layers and platform-level controls.

OWASP 2025 risk	Most exposed layers	Platform-level controls
LLM01 Prompt Injection	Knowledge, agent and tool, memory	Separate trusted and untrusted content; restrict tools for agents reading untrusted input; approval gates
LLM02 Sensitive Information Disclosure	Knowledge, memory, model access	Permission-filtered retrieval; masking; retention rules; provider data agreements
LLM03 Supply Chain	Model access, agent and tool	Approved model and tool registry; version pinning; review of third-party servers
LLM05 Improper Output Handling	Agent and tool, orchestration	Schema validation of tool arguments; treat model output as untrusted input
LLM06 Excessive Agency	Agent and tool, identity	Least-privilege tool sets; delegated tokens; limits on amounts and scope
LLM08 Vector and Embedding Weaknesses	Knowledge	Access control on indexes; provenance; ingestion screening
LLM10 Unbounded Consumption	Model access, cost control	Quotas, budgets, rate limits, loop and step limits

Risk identifiers and names from [24]; layer mapping and controls by the author.

Because prompt injection cannot currently be prevented reliably at the model level, the decisive control is architectural: an agent that reads untrusted content should not simultaneously hold tools that can move money, change records or send data outside the organization.

6. Maturity Model and Phased Adoption

Organizations rarely build all layers of Table 1 at once. Table 3 proposes a five-stage maturity model derived from the architecture. The stages are cumulative and describe capabilities rather than elapsed time; an organization may be at different stages for different capabilities. The model is a planning aid proposed in this paper, not an empirically validated instrument.

Table 3. Proposed five-stage maturity model for an enterprise AI operating system.

Stage	Characteristics	Capabilities added	Exit criteria
0 Ad hoc	Individual use of public assistants; no inventory	None shared	Acceptable-use policy issued; usage inventoried
1 Managed access	Sanctioned assistant and model gateway	Single sign-on, gateway logging, data-handling rules	All model traffic passes through gateway
2 Grounded assistance	Retrieval over internal content	Permission-filtered knowledge layer, evaluation suite	Retrieval quality measured; no permission leakage in tests
3 Supervised automation	Workflows with tools and human approval	Tool registry, orchestration, delegated identity, cost attribution	Eq. (2)–(4) reported per workflow
4 Governed autonomy	Agents act within policy without per-step approval	Policy engine, continuous evaluation, incident response, management system	Audit against chosen framework passed

Proposed by the author; framework alignment with [17, 19].

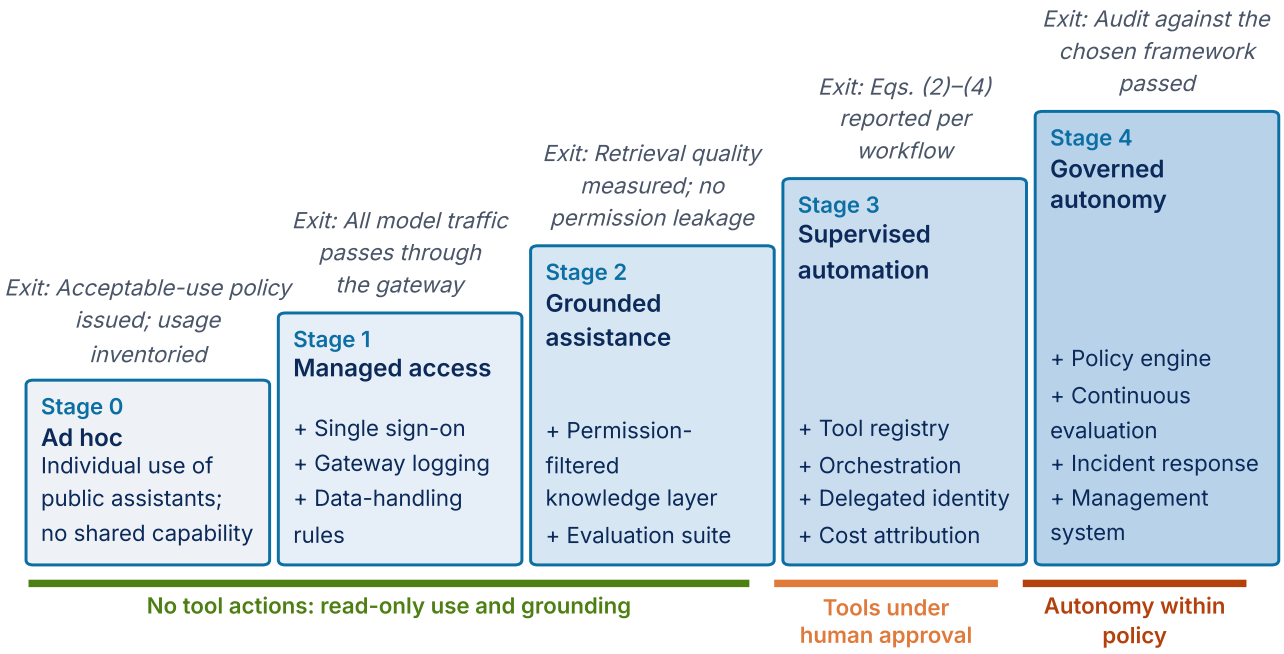


Figure 2. The proposed five-stage maturity model of Table 3 as a staircase. Each step lists the capabilities added at that stage and, above it, the exit criterion for moving to the next stage; the bands below mark where tool actions and autonomy are introduced.

Drawn by the author from Table 3.

The progression reflects a deliberate ordering. Managed access comes first because it provides the inventory and logging on which all later governance depends. Retrieval precedes tool use because read-only grounding carries lower risk than write actions and allows the organization to build evaluation capability before automation raises the stakes. Supervised automation introduces tools under human approval, which generates labeled data on agent decisions that later justifies relaxing approval for low-risk actions. Governed autonomy is reached only for workflows whose measured reliability, as in Eq. (4), and risk classification warrant it. This sequencing aligns with the iterative Govern–Map–Measure–Manage cycle of the NIST framework [17] and the continual-improvement structure of ISO/IEC 42001 [19]. Figure 2 draws Table 3 as a staircase: each stage adds capabilities to those below it, and tool actions appear only at Stage 3, after inventory, grounding and evaluation are in place.

7. Illustrative SME Scenario

This section applies the metrics to a hypothetical example. All figures are assumptions chosen for illustration; they do not describe any real organization, and the prices are not quotations

from any provider. Consider a wholesale distributor with about 40 staff that processes 2,000 supplier invoices per month and has reached Stage 3 of Table 3 for invoice processing. Each invoice passes through a four-step workflow: document intake and classification, field extraction by a large model, validation against purchase orders by deterministic code assisted by a small model, and posting to the accounting system after any required human approval.

Assume that extraction consumes 6,000 input and 500 output tokens on a large model priced at $3 and $15 per million input and output tokens, giving $0.018 + $0.0075 = $0.0255 per invoice, and that validation consumes 2,000 input and 200 output tokens on a small model priced at $0.25 and $1.25 per million tokens, giving $0.00075. Tool and infrastructure costs are assumed to be $0.005 per invoice. Assume further that 15% of invoices are escalated for human review, each requiring 3 minutes of a clerk costed at $40 per hour, or $2.00 per escalated invoice and $0.30 averaged over all invoices. If 97% of invoices are completed correctly, Eq. (2) gives C_{task} = ($0.0255 + $0.00075 + $0.005 + $0.30) / 0.97 ≈ $0.34 per invoice. Model tokens account for less than 8% of this cost; human review accounts for nearly 90%.

With 85% of invoices completed without intervention, Eq. (3) gives A = 0.85. Suppose the step reliabilities are 0.99, 0.98, 0.97 and 0.99. Eq. (4) gives R = 0.99 × 0.98 × 0.97 × 0.99 ≈ 0.932 for unattended operation. If the validation step is allowed a second independent attempt when a checksum fails, its effective reliability becomes $1 - 0.03^2 = 0.9991$ and R rises to approximately 0.960, subject to the independence caveat in Section 4.3. Under these assumptions, the most effective improvement is not a cheaper model but a reduction in the escalation rate through better extraction on difficult layouts, since each percentage point of escalation removed saves about $0.02 per invoice, roughly the entire large-model cost.

The same scenario illustrates the governance controls. The extraction agent reads untrusted supplier documents and therefore holds no write-capable tools; posting is performed by deterministic code after validation and, above a value threshold, after human approval under the approver's own delegated identity. The workflow is registered in the inventory with an owner, an evaluation suite of labeled historical invoices and a documented model version, so that a model upgrade triggers re-evaluation before deployment. Figure 3 brings the workflow, its controls and the metrics of this section into one view, and shows that the human-review step, which is not part of the reliability product, carries most of the cost per invoice.

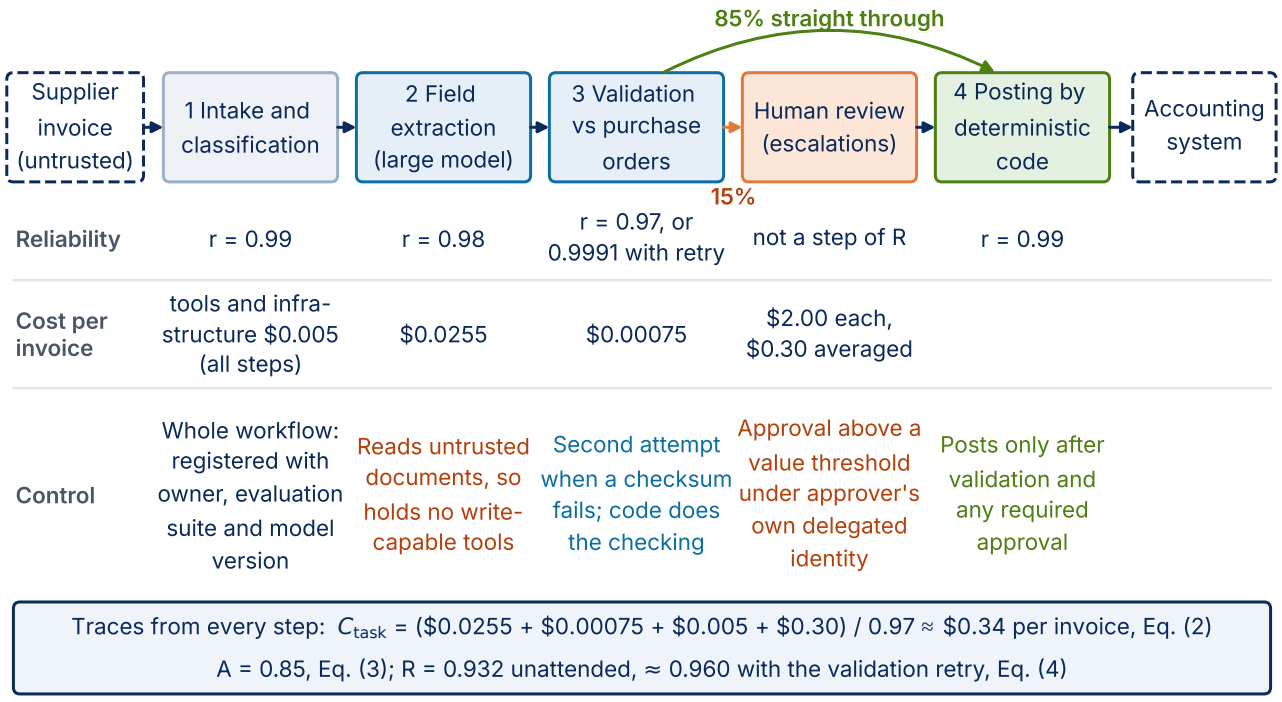


Figure 3. The hypothetical invoice workflow of Section 7 with its step reliabilities, costs per invoice and controls. Eighty-five percent of invoices pass straight through and 15% are escalated for review; all values are the illustrative assumptions of this section.

Author's calculation from Eqs. (2)–(4) with the hypothetical inputs stated in this section.

8. Limitations and Open Problems

The architecture and maturity model in this paper are proposals synthesized from the literature and standards; they have not been validated empirically across a population of organizations. Several problems remain open. First, prompt injection lacks a general defense, and the structural mitigations of Section 5 reduce capability as well as risk [26, 24]. Second, evaluation of agentic workflows remains costly, and automated judges carry documented biases [23]. Third, routers and cascades depend on quality predictors whose calibration may degrade when models or traffic change [15, 16]. Fourth, the regulatory landscape continues to evolve; organizations should consult the current consolidated text of applicable instruments rather than secondary summaries [20]. Finally, interoperability standards such as MCP are recent, and their security model relies on implementers to enforce consent and authorization [13].

There is also a risk of over-engineering. An SME with two AI workflows does not need every layer of Table 1 implemented as separate infrastructure; the layers are logical responsibilities

that may be fulfilled by a small number of services or by configuration of commercial platforms. The value of the reference architecture lies in making each responsibility explicit so that none is omitted by default.

9. Conclusion

As AI deployments multiply, organizations face a choice between a fragmented estate of independent assistants and agents and a shared layer that provides common services for data, knowledge, models, tools, workflows, memory, identity, governance, evaluation and cost. This paper has described the latter under the name AI operating system, building on research that treats LLM runtimes as operating systems [2, 3], on tool-use and retrieval research [5, 6, 9], on the Model Context Protocol as an interoperability standard [13], and on governance and security frameworks [17, 19, 24].

Three conclusions follow. First, mediation of all model, tool and data access through platform services is the architectural decision on which governance, security and cost control depend. Second, the economic performance of an AI OS should be judged by cost per successfully completed task and by measured reliability, not by token prices, since human review and failure frequently dominate total cost. Third, adoption should proceed in stages that build inventory, grounding and evaluation capability before granting agents autonomy over consequential actions.

> An AI operating system is best understood as a set of explicit responsibilities rather than a product: every model call, tool action and retrieved document should pass through a layer that knows who requested it, whether it is permitted, what it cost and whether it was correct.

References

1. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. Advances in Neural Information Processing Systems, 28. papers.nips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abst
2. Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. arXiv preprint arXiv:2310.08560.

arxiv.org/abs/2310.08560

3. Mei, K., Zhu, X., Xu, W., Hua, W., Jin, M., Li, Z., Xu, S., Ye, R., Ge, Y., & Zhang, Y. (2024). AIOS: LLM agent operating system. arXiv preprint arXiv:2403.16971 (published at COLM 2025). arxiv.org/abs/2403.16971
4. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J.-R. (2023). A survey on large language model based autonomous agents. arXiv preprint arXiv:2308.11432. arxiv.org/abs/2308.11432
5. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations (ICLR). arXiv:2210.03629. arxiv.org/abs/2210.03629
6. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761. arxiv.org/abs/2302.04761
7. Karpas, E., Abend, O., Belinkov, Y., Lenz, B., Lieber, O., Ratner, N., Shoham, Y., Bata, H., Levine, Y., Leyton-Brown, K., et al. (2022). MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. arXiv preprint arXiv:2205.00445. arxiv.org/abs/2205.00445
8. Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2023). Gorilla: Large language model connected with massive APIs. arXiv preprint arXiv:2305.15334. arxiv.org/abs/2305.15334
9. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33. arXiv:2005.11401. arxiv.org/abs/2005.11401
10. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997. arxiv.org/abs/2312.10997
11. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics, 12. arXiv:2307.03172. arxiv.org/abs/2307.03172
12. Anthropic. (2024, November 25). Introducing the Model Context Protocol. Anthropic News. www.anthropic.com/news/model-context-protocol
13. Model Context Protocol. (2025). Specification, version 2025-06-18. modelcontextprotocol.io. modelcontextprotocol.io/specification/2025-06-18
14. Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero trust architecture (NIST Special Publication 800-207). National Institute of Standards and Technology.

10.6028/NIST.SP.800-207

15. Chen, L., Zaharia, M., & Zou, J. (2023). FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv preprint arXiv:2305.05176. arxiv.org/abs/2305.05176
16. Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2024). RouteLLM: Learning to route LLMs with preference data. arXiv preprint arXiv:2406.18665. arxiv.org/abs/2406.18665
17. National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0) (NIST AI 100-1). 10.6028/NIST.AI.100-1
18. National Institute of Standards and Technology. (2024). Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1). 10.6028/NIST.AI.600-1
19. International Organization for Standardization & International Electrotechnical Commission. (2023). ISO/IEC 42001:2023 Information technology — Artificial intelligence — Management system (1st ed.). webstore.iec.ch/publication/90574
20. European Parliament & Council of the European Union. (2024). Regulation (EU) 2024/1689 of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, L series. eur-lex.europa.eu/eli/reg/2024/1689/oj
21. Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19), 220–229. 10.1145/3287560.3287596
22. Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., et al. (2023). Holistic evaluation of language models. Transactions on Machine Learning Research. arXiv:2211.09110. arxiv.org/abs/2211.09110
23. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track. arXiv:2306.05685. arxiv.org/abs/2306.05685
24. OWASP Foundation. (2024). OWASP Top 10 for LLM Applications 2025. OWASP Gen AI Security Project. genai.owasp.org/llm-top-10/
25. Perez, F., & Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. NeurIPS ML Safety Workshop. arXiv:2211.09527. arxiv.org/abs/2211.09527

26. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), 79–90. 10.1145/3605764.3623985

How to cite this article

Lee, Q. F. (2026). AI Operating Systems: A Reference Architecture and Governance Model for Enterprise AI. *Chmltech Online Journal*, *104*(1). https://www.chmltech.com/post/ai-operating-systems-architecture-and-governance-for-enterprise-ai

Ready to find your next improvement?

Book a strategy call. We will talk through your goals, pain points and where AI, systems or process work can make the biggest difference.

Book a Strategy Call

Or email **info@chmltech.com**